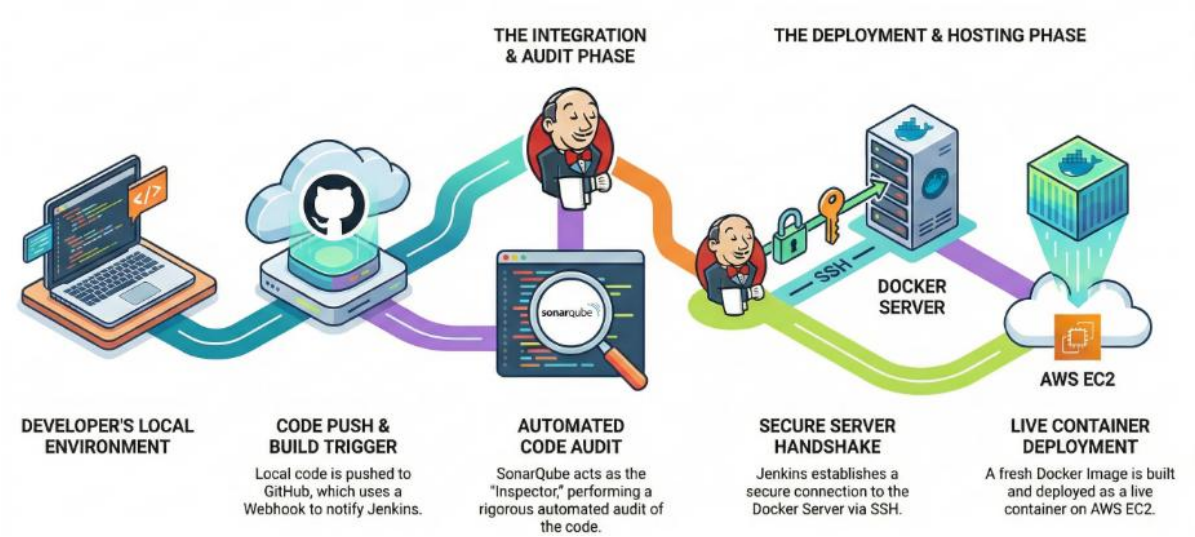
Code to Cloud: The Pipeline

Commit. Conquer. Deploy. Because "Manual" is just another word for "Mistake."



Authored By

Shehar Bano

Ref

v1.0 | 2026.02

CI/CD Pipeline: Concept and Workflow

What is a Pipeline?

A **Pipeline** is an automated sequence of steps that takes code from a developer's machine to a production environment. Its purpose is to **standardize the delivery process**, ensuring that every change is automatically built, tested for quality, and deployed without manual intervention.

Our Tech Stack

To build this robust automation, we are integrating:

- **GitHub:** Source Control (Where the code lives).
- **Jenkins:** The "Brain" (Orchestrates all movements).
- **SonarQube:** The "Inspector" (Checks code quality and security).
- **Docker:** The "Package" (Ensures the app runs everywhere).

The Workflow: From Terminal to Deployment

➢ Code Push via Terminal

The developer completes a feature and executes git push from their local terminal to the **GitHub** repository.

➢ Webhook Trigger

GitHub sends a signal (Webhook) to Jenkins, notifying it that new code is available for processing.

➢ Continuous Integration & Build

Jenkins pulls the latest code and begins the build process to ensure the application compiles correctly.

➢ Automated Code Audit

The code is sent to **SonarQube**. It scans for "code smells," bugs, and security vulnerabilities. If the quality gate fails, the pipeline stops.

➢ Environment Cleanup

To prepare for the new version, Jenkins executes commands to safely stop and remove the existing container (my-site). This ensures there are no port conflicts during the update.

➢ Local Containerization

Once the environment is cleared, **Jenkins** uses the **Dockerfile** to build a fresh **Docker Image** locally on the host machine. This packages the updated application and its dependencies into a single unit.

➢ Automated Deployment & Execution

In the final step, **Jenkins** immediately runs the new Docker container from the local image, mapping the necessary ports (80:80) to make the application live instantly.

May your builds be green and your latency low

Why Use Local Images vs. Docker Hub?

Feature	Local Image (Our Method)	Docker Hub
Speed	**Fastest** (Local Disk)	**Slower** (Network dependent)
Privacy	**Highest** (Never leaves host)	**Variable** (External storage)
Connectivity	Works offline	Requires Internet
Complexity	Simple `docker build`	Complex (Auth + Push/Pull)

Implementation

To implement the CI/CD pipeline, we will use **AWS (Amazon Web Services)** as our cloud provider.

What is AWS?

AWS is a comprehensive cloud platform provided by Amazon that offers over 200 fully-featured services (like servers, storage, and databases) on a "pay-as-you-go" basis. Its purpose is to allow businesses and developers to obtain large-scale computing capacity quickly and cheaply without having to build and maintain physical hardware.

Infrastructure Setup: EC2 Instances

We will use **EC2 (Elastic Compute Cloud)** instances, which are essentially virtual servers in the cloud, to host our tools. For this pipeline, we will launch **3 separate instances**:

Tool	Operating System	Instance Type	Reason for Selection
Jenkins	Ubuntu	c7i-flex.large	**Compute-Optimized:** Jenkins handles heavy building and orchestration tasks. The c7i-flex provides the best performance-to-price ratio for compute-intensive workloads.
SonarQube	Ubuntu	m7i-flex.large	**General Purpose:** SonarQube requires a balance of CPU and high RAM to scan code and manage its internal database. The m7i series offers the necessary memory-to-vCPU ratio.
Docker	Ubuntu	t3.micro	**Lightweight:** Since this is our deployment host for a basic website, a t3.micro is sufficient, cost-effective, and often eligible for the AWS Free Tier.

Step 1: Assigning Ports (Security Groups)

By default, AWS blocks all traffic to your instances for security. To see your tools running, you must open specific "doors" called **Ports**.
Port Mapping

- **Jenkins (8080):** The default port where the Jenkins dashboard is hosted.
- **SonarQube (9000):** The default port for the SonarQube web interface.
- **Docker (80/443):** Standard HTTP/HTTPS ports used to make your website accessible to the public.

How to Assign Ports

Go to the **EC2 Dashboard** and select your instance.

1. Click on the **Security** tab and select the **Security Groups** link.
2. Click **Edit inbound rules**.
3. Click **Add rule**:
   - **Type:** Custom TCP
   - **Port Range:** Enter your port (e.g., 8080)
   - **Source:** Select Anywhere-IPv4 (0.0.0.0/0) for testing.
4. Click **Save rules**.

Tip: Once saved, copy the **Public IP** of your instance, paste it into your browser, and add the port at the end (e.g., 3.14.21.5:8080) to access your tool.

Step 2: Connecting to Your Instances

You have two main ways to talk to your virtual machines: using your own computer's Terminal (via SSH) or using the **AWS Browser-based Console**.
Why we use the Browser-based Connection:

- **Convenience:** No need to download or manage SSH .pem keys on your local machine.
- **No Configuration:** It works instantly regardless of your computer's operating system (Windows, Mac, or Linux).
- **Security:** It uses IAM permissions directly, so you don't have to worry about losing private key files.

Connection Steps:

1. Click on your **Jenkins** instance from the list.
2. Click the **Connect** button at the top right.
3. Select the **EC2 Instance Connect** tab.
4. Click **Connect**. A new terminal tab will open in your browser, and your machine is ready to use!
5. **Repeat** these steps for the SonarQube and Docker instances.

Now, let's get into the heavy lifting. We are going to install our tools and make them talk to each other.

1. Setting up the Jenkins Instance ("The Brain")

First, update the package manager to ensure we pull the latest versions.

Bash

```
sudo apt update
```

Install Java Runtime (JRE)

Reason: Jenkins is a Java-based application. It requires the Java Virtual Machine (JVM) to run its core engine.

Bash

```
sudo apt install openjdk-17-jre
```

Install Jenkins

We add the repository key and the software list to Ubuntu, then install the tool.

Bash

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install Jenkins
```

Verify: Check if it's running: `systemctl status jenkins`

- **Access:** Open your browser and go to `http://<Jenkins_IP>:8080`
- **Unlock:** To get the initial password, run: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
- **Setup:** Follow the prompts to "Install Suggested Plugins" and create your admin account.

Install Node.js

Reason: SonarQube requires a Node.js runtime to power its specialized "sensors" for JavaScript/TypeScript. Without it, the "Inspector" cannot perform deep security audits or logic checks on your code.

Bash

```
sudo apt update && sudo apt install nodejs -y
```

2. Setting up the SonarQube Instance ("The Inspector")

Update the system: `sudo apt update`

Install Java Runtime

Reason: Like Jenkins, SonarQube is built on Java and requires it for code analysis processing.

Bash

```
sudo apt install openjdk-17-jre
```

Install SonarQube

Bash

```
sudo apt install unzip

wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-
9.9.0.65466.zip

unzip sonarqube-9.9.0.65466.zip

cd sonarqube-9.9.0.65466.zip/bin/linux-x86-64
```

- **Why `linux-x86-64`?** This folder contains the specific binaries for 64-bit Linux operating systems (which our EC2 instance is).

- **Start the tool:** `./sonar.sh start` (This script triggers the background Java process).

Access: Go to `http://<SonarQube_IP>:9000` Login with **admin/admin** and update the password.

3. Setting up the Docker Instance ("The Package")

Update the system: `sudo apt update`

Install Docker

Bash

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Verify Docker Installation: `sudo docker –version`

4. Integrating SonarQube and Jenkins

In SonarQube Dashboard:

1. **Manual Project:** Choose "Manual" because it gives us full control over the configuration settings without needing pre-existing GitHub Apps.
2. **Setup:** Name it `my-website`. Select **Jenkins** as the CI tool (tells Sonar where the reports are coming from) and **GitHub** as the platform (where the code lives).
3. **Generate Token:** Go to **Avatar > My Account > Security**.
    - **Reason for Global Token:** A global token allows Jenkins to use this single credential to scan any project on this SonarQube server.
    - **Generate and Save** this token immediately!

In Jenkins Dashboard:

1. **Plugins:** Go to **Manage Jenkins > Plugins > Available**. Install **SonarQube Scanner** and **Publish Over SSH**.

2. **Security Fix:** Under **Manage Jenkins > Security**, check **Enable proxy compatibility** for the Crumb Issuer.
    - **Reason:** This prevents "403 Forbidden" errors when Jenkins is behind a load balancer or proxy.
3. **Add Credentials:** Go to **Manage Jenkins > Credentials**. Add a "Secret Text" credential. Paste your Sonar token into the **Secret** field and ID it as `shehar-token`.
4. **Configure System:** Go to **Manage Jenkins > System**. Find "SonarQube Servers", add the Server URL (`http://<Sonar_IP>:9000`), and select the `shehar-token` credential.
    - **Reason:** This "binds" the two servers so Jenkins can automatically push scan results to Sonar.

5. Connecting Jenkins to Docker (SSH Handshake)

We need Jenkins to "log in" to the Docker server to run commands.

On Jenkins Server (Terminal):

Reason for SSH Key: We use RSA keys so Jenkins can log into the Docker server securely without needing a password.

Bash

```
ssh-keygen -t rsa -b 4096  # Hit Enter 3 times to skip passphrase/default
path
cat ~/.ssh/id_rsa.pub     # This is your "Lock" - Copy it
```

On Docker Server (Terminal):

Bash

```
mkdir -p ~/.ssh
nano ~/.ssh/authorized_keys # Paste the key from Jenkins here
chmod 700 ~/.ssh            # Secure the directory
chmod 600 ~/.ssh/authorized_keys # Secure the key file
```

Reason for Chmod: SSH will reject keys if they are "too open" (readable by others). These commands lock the permissions.

Back on Jenkins Dashboard:

1. **Copy Private Key:** On Jenkins terminal, run `cat ~/.ssh/id_rsa` and copy the text.
2. **Add SSH Server:** Go to **Manage Jenkins > System > SSH Servers**.
    - **Key:** Paste the private key here (the "Master Key").
    - **Hostname:** `docker-server`
    - **Username:** `ubuntu`.
3. **Test Configuration:** Click the button. If it says **Success**, Jenkins now officially "owns" the Docker server.
4. Click **Save** and you'll be back to Jenkins Home Page.

Now, we are going to create the actual "Track" for our automation train to run on. We will build a **Freestyle Pipeline**, which is the most user-friendly way to visualize the steps.

1. Creating the Pipeline

1. On the Jenkins Dashboard, click **New Item**.
2. Enter the name `shehars-pipeline`.
3. Select **Freestyle project**.
    - **Reason:** It provides a simple, graphical interface to configure steps without needing to write complex code (Groovy).

In CI/CD, Groovy's role is to provide **"Pipeline as Code."**
It is the scripting language used to write a **Jenkinsfile**, which replaces manual dashboard clicks with a version-controlled script.
It's Role in 3 Points:

- **Standardization:** It defines your entire Build-Test-Deploy sequence in a single text file.
- **Logic:** It handles "if/then" decisions (e.g., *"Only deploy if SonarQube passes"*).
- **Portability:** You can move your pipeline to a new Jenkins server instantly by simply loading the script from GitHub.

4. Click **OK**.

2. Source Code Management (SCM)

1. In the **General** section, find **Source Code Management** and select **Git**.
2. **Repository URL:** Paste your GitHub repo link.
    - **Reason:** This tells Jenkins exactly where to "fetch" the code from whenever an update happens.
3. **Branch Specifier:** Change `*/master` to `*/main`.
    - **Reason:** Modern GitHub repositories use `main` as the default primary branch.
4. **Build Triggers:** Check the box for **GitHub hook trigger for GITScm polling**.
    - **Reason:** This enables "Push-button" automation; Jenkins will start the build the moment it receives a signal from GitHub.

4. The Build Steps (Analysis & Deployment)

Step A: SonarQube Analysis

In the **Build Steps** section, click **Add build step** and choose **Execute SonarQube Scanner**. In **Analysis properties**, paste:

Properties

```
sonar.projectKey=my-website
sonar.sources=.
```

- **Reason:** `projectKey` links this scan to your SonarQube project, and `sources=.` tells the scanner to inspect every file in the current folder.

Step B: Deployment via SSH

Click **Add post-build action** and choose **Send build artifacts over SSH**.

- **Reason:** This allows Jenkins to "reach out" to the Docker server and run the final deployment commands.

Configure the Transfer:

- **Source files:** `**` (**Reason:** This uses a **glob pattern** to recursively include all files and subdirectories from the Jenkins workspace.).
- **Remote directory:** `Pipeline` (**Reason:** A dedicated folder on the Docker server to keep the project organized).
- **Exec command:** Paste the following:

Bash

```
set -e
cd /home/ubuntu/Pipeline
sudo docker stop my-site || true
sudo docker rm -f my-site || true
sudo docker build -f my-app/Dockerfile -t my-website-image:latest .
sudo docker run -d -p 80:3000 --name my-site my-website-image:latest
```

Reason: These commands stop the old version of your site, build a fresh image from the new code, and start the new container by mapping the public **port 80** to the application's internal **port 3000**.

4. Final Link: The GitHub Webhook

1. Copy your Jenkins URL: `http://<Jenkins-IP>:8080/`
2. Go to your **GitHub Repo > Settings > Webhooks > Add webhook**.
3. **Payload URL:** `http://<Jenkins-IP>:8080/github-webhook/` (The trailing slash is important!).
4. **Content type:** `application/json`.
5. Click **Add webhook**.

5. The Moment of Truth

1. Go back to Jenkins and click **Build Now** to test it manually.
2. Once the ball turns green (Success!), copy your **Docker Instance IP** and paste it into your browser.
3. **Your website is now LIVE!**

What's next? Try changing a single word in your code on your local machine and run `git push`. If everything is set up correctly, your website will update itself in less than a minute!

Resource Cleanup

To wrap up your project and stop AWS billing, clear these four items:

- **Terminate EC2 Instances:** This stops hourly charges for the **Jenkins** (c7i-flex), **SonarQube** (m7i-flex), and **Docker** (t3.micro) servers.
- **Delete Security Groups:** This removes the specific "doors" opened for ports **8080**, **9000**, and **80**.
- **Release Elastic IPs:** This prevents AWS from charging you for reserved addresses that are no longer attached to active servers.
- **Delete Key Pairs:** This clears the RSA keys used for the **SSH Handshake** to maintain clean security hygiene.

May your builds be green and your latency low

Impact of Website's Language Change on Pipeline

1. The Build Context (Dockerfile)

- **Change:** You must update the `FROM` instruction (e.g., from `node:alpine` to `python:3.9` or `openjdk:17`).
- **Reason:** Different languages require different **runtimes and compilers**. A Python app cannot run in a Node.js environment; the Dockerfile ensures the remote server has the specific "engine" needed for that language.

2. Dependency Management

- **Change:** The command to install libraries changes (e.g., `npm install` becomes `pip install -r requirements.txt` or `mvn clean install`).
- **Reason:** Each language has its own **package manager**. To keep the deployment automated, the `Exec command` or Dockerfile must tell the server how to fetch the specific libraries that the new language requires.

3. Port Mapping & Internal Exposure

- **Change:** The internal port in the `docker run` command might change (e.g., `-p 80:5000` for Python/Flask instead of `-p 80:3000` for Node.js).
- **Reason:** Different frameworks have different **default listening ports**. If you're new language's framework defaults to port 5000, your pipeline must be updated to map that specific internal port to the public port 80.

Comparison Table: Pipeline Variations

Feature	Node.js (Current)	Python (Flask)	Java (Spring Boot)
Base Image	`node:18`	`python:3.10`	`openjdk:17`
Build Command	`npm install`	`pip install`	`mvn package`
Default Port	`3000`	`5000`	`8080`
Exec Command	`docker run -p 80:3000`	`docker run -p 80:5000`	`docker run -p 80:8080`

Note: The "Source Files" (**) and "Remote Directory" (Pipeline) settings usually **do not change**, because regardless of the language, you still need to send all files to a specific folder on your server.